

Node.js Interview Questions & Answers

Middleware in Node.js

Middleware are functions in Express.js that sit between the request and response. They can modify req/res objects, end the request-response cycle, or call next() to pass control to the next middleware. Types include: Application-level, Router-level, Built-in, Error-handling, and Third-party middleware.

Use of next() function in Middleware

The next() function passes control to the next middleware or route handler. If next() is not called, the request will hang. Variants: next() → go to next middleware, next('route') → skip to next route, next(err) → skip to error-handling middleware.

Callback Hell in Node.js and How to Avoid It

Callback hell occurs when multiple async callbacks are nested, making code unreadable. Example: deeply nested fs.readFile calls. Avoid using: Modularizing callbacks, Promises, and Async/Await (best modern approach).

Difference between Promise and Async/Await

Promises use .then() and .catch() chaining. Async/Await is syntactic sugar over Promises, making async code look synchronous and easier to read. Promises can become messy with chaining, while async/await improves readability using try...catch.

How Can You Scale a Node App?

1. Use Clustering to utilize all CPU cores. 2. Load balancing across servers with NGINX/Kubernetes. 3. Use PM2 process manager. 4. Optimize DB + caching (Redis). 5. Use queues (RabbitMQ, Kafka) for heavy tasks. 6. Adopt microservices. 7. Containerization with Docker + orchestration with K8s. 8. Monitoring & logging with Prometheus, ELK.

JWT (JSON Web Token) in Detail

JWT is a compact, URL-safe token format used for authentication & authorization. Structure: Header (alg, typ), Payload (claims like id, role, exp), Signature (verify integrity). Flow: User logs in → Server creates token → Client stores it → Client sends token in Authorization header → Server verifies it. Advantages: Stateless, cross-platform, fast, built-in expiry & claims, scalable. Best practices: short expiry, use HTTPS, store in HTTP-only cookies, use refresh tokens.

Advantages of JWT for Authentication & Authorization

1. Stateless → No server-side session storage. 2. Cross-platform → Works across languages. 3. Compact & fast. 4. Built-in expiry & claims. 5. Scalable across multiple servers. 6. Secure (signed, tamper-proof). 7. Supports role & permission-based access easily.

Event Loop: `process.nextTick()` vs `setImmediate()`

`process.nextTick()` callbacks run after current synchronous code, before event loop continues.

`setImmediate()` callbacks run in the check phase. Execution order: 1. Sync code → 'Program Started' 2. `process.nextTick()` → '1st Process', '2nd Process' 3. `setImmediate()` → '1st Immediate', '2nd Immediate'